Yaad — Features & Differentiators

What we're building, and what makes it different from "a chatbot with a voice"

> **How this fits the other docs:** `CONTRACT.md` = the exact API you must match · **this file = what each feature must DO and why it's novel** · `CLAUDE.md` = full architecture per track · `PLAN.md` = the 24h schedule · `STATUS.md` = live build log. **Claude Code: read CONTRACT.md → this file → your `packages/<track>/CLAUDE.md` → PLAN.md.** Build the ★ items so they are *unmistakable* — they are what wins.

The one-sentence difference

Yaad is **not** ChatGPT with a microphone. It's an **episodic memory graph built on Moss** that gives a person *their own life* back — grounded, temporal, and updated live. The novel surface area is the **memory layer**, not the conversation.

★ THE UNIQUE THINGS (the differentiators — build these to shine)

Every one of these is a thing the obvious version (a RAG chatbot) does *not* do. These are the demo and the moat.

★1. Episodic memory graph on Moss — *not flat RAG*

Memory is a **typed graph**: entities (person/place/event/medication/story) + **episodes** + **edges** (relationships), traversable 1 hop during retrieval. - **Novel:** every other "AI memory" demo is a vector blob. Ours is a *structured life* you can walk — "who is Leo?" pulls Leo *and* the episodes/people connected to him. - **How:** `graph.py` (entities/episodes/edges + 1-hop traversal); edges carry weights; retrieval adds a graph-proximity term. - **Surface:** `POST /memory/query` returns connected items, not just nearest neighbors. - **Done:** asking about one person surfaces their relationships and shared episodes, ranked.

★2. Composed retrieval scoring — *recall like a human, not like a search bar*

`score = α·semantic + β·recency + γ·salience + δ·graph_proximity`, with `recency = exp(−λ·Δt)`. - **Novel:** relevance isn't raw cosine similarity. Recent, emotionally salient, and *connected* memories surface first — the way human recall actually works. - **How:** `retrieval.py`; tune α/β/γ/δ on the seed data; expose the components for the demo. - **Surface:** every `RetrievedItem` carries its `score`. - **Done:** a recent salient memory outranks an older, more semantically-similar one.

★3. Temporal state reasoning — *memory that knows "when"*

"Did I take my pills today?" routes to **today's** `med_log`, not a semantic match on the word "pills." - **Novel:** this is the single most important question a person with dementia asks, and a vanilla RAG bot answers it wrong. Yaad reasons over time. - **How:** `temporal.py` time-intent router; "today/yesterday/is X coming" → med_log / upcoming events with date math. - **Surface:** `POST /memory/temporal`. - **Done:** log a dose, then "did I take my pills today?" → "Yes, at 8 this morning." Before logging → "Not yet today."

★4. Strict grounding / anti-confabulation — *it refuses to make up your life*

Assert **only** retrieved facts. Below a confidence threshold → "I'm not sure, let me check with the family." Every claim carries **provenance** (who added it, when, source). - **Novel:** the moment the AI *declines to invent* a person or memory is the trust moment of the whole demo. Every other companion hallucinates a fake life — which, for a dementia patient, is harmful. - **How:** `grounding.py` confidence gate τ → safe-refusal draft; provenance attached to every item; auditable. - **Surface:** `/memory/query` returns `grounded` + `confidence`; if `grounded=false`, `answer_draft` is the safe refusal. - **Done:** ask about a person who isn't in memory → it says it doesn't know and offers to check, and **never** fabricates.

★5. Instant, on-device updates — *Moss's actual superpower, on stage*

Family adds a fact → it's usable in the **next sentence** (**add-fact-live**). And it works **wifi-off** (on-device Moss index). - **Novel:** live index updates with no re-embedding lag is exactly what Moss exists to do — and the two beats (add-a-fact-live, then pull the plug and it still answers) are the signature moments judges remember. - **How:** `/memory/write` writes to Supabase **and** upserts into Moss instantly; on-device/WASM mode for wifi-off [CONFIRM at office hours]. - **Surface:** `POST /memory/write`. - **Done:** type "Leo's birthday is Saturday" in the web app → ask the agent 5 seconds later → it knows. Turn off wifi → the 5 core beats still answer.

★6. Cross-lingual retrieval — *the memory is language-agnostic*

A **Hindi** question retrieves a memory **stored in English** (and answers in Hindi). - **Novel:** this is *not* a translated voice. The memory layer is multilingual at the embedding level — ask "Leo kaun hai?" and it recalls the English-stored fact about Leo. - **How:** MiniMax multilingual TTS + language detect on the transcript + cross-lingual embeddings in Moss [CONFIRM]. - **Surface:** `/memory/query {text, lang}`. - **Done:** Hindi question → correct memory retrieved → warm Hindi answer. (This is the technical flex — make the demo show cross-lingual *retrieval*, not just a Hindi voice.)

CORE FEATURES (the working product — ★ build first, must never fail)

- **Real-time voice loop** — LiveKit + Pipecat transport, VAD, Deepgram STT, intent/lang routing, TrueFoundry LLM with the grounding prompt, MiniMax TTS, **barge-in**, speculative retrieval on partial transcript → <~1s. *Done: "who is Leo?" warm + grounded under ~1s; interrupting stops the TTS.*
- **The 5 demo beats** (each is a feature): who-is-this · pills-today (★3) · add-fact-live (★5) · wifi-off (★5) · Hindi exchange (★6).
- **Caregiver web — add-memory forms** — person/event/med/story, one-click fast. *This is the engine of add-fact-live.* → `/memory/write`.
- **Seed (`seed_amma.py`)** — a rich, believable life for "Amma" (84; grandson Leo, daughter Sarah, meds, routine, episodes, home + a familiar park) so the demo feels real.

SUPPORTING FEATURES (build after core is flawless)

- **Memory graph visualizer** — force-directed graph (`MemoryGraph.tsx`) so judges *see* the graph is real, not flat RAG.
- **Timeline reconstruction** — "what did I do yesterday?" → ordered blocks. → `/memory/timeline`.
- **Care dashboard** — "topics to reinforce with her" (caregiver guidance). **NO fake clinical/health score** (see NOT-building).
- **Proactive reminders** — at a med time / before a visit, the agent speaks first ("It's 8 — your white heart pill; Sarah visits at 3"). → `/reminders/due`.
- **Autonomous capture** — explicit "remember this..." → extract entities/episode → store. *Honest scope: ship the explicit-trigger + caregiver-review version; the reliable live-update beat is the web form, not silent auto-capture.* → `/memory/capture`.
- **Document ingestion (Unsiloed)** — upload one medical letter → parsed into structured memory (meds/appointments). *This is the Unsiloed sponsor flex — give it one concrete task or drop it from the sponsor thanks.*

OPTIONAL (pick exactly ONE at Gate 3 — recommendation: Vision)

- **Vision (recommended)** — snapshot on command → recognize a **pre-registered** face → "that's Leo, your grandson." *Single-shot, never continuous.* Trivial fixture fallback (one photo → one person ref). → `/vision/recognize`.
- **Wander-safety** — leaves the safe-zone or says she's lost → **reassure + alert a human with her location** (Twilio/push). *Reassure + keep-in-place + alert. NEVER navigate.* → `/location/ping`.

✗ EXPLICITLY NOT BUILDING (so the agent doesn't drift into these)

- ✗ **Real-time street/traffic navigation for a disoriented person.** Can't be done safely at conversational latency; failure mode is someone getting hurt. Lost → reassure + alert a human instead. *Hard guardrail in code and pitch.*
- ✗ **A "memory health score" / cognitive-decline heatmap.** It would measure topic frequency, not cognition — fake-clinical and misleading. Use plain "topics to reinforce with her."
- ✗ **Correcting or contradicting the patient.** Clinically wrong for dementia (validation > correction). Discrepancies become a *silent caregiver-side note only*, never a correction to her.
- ✗ **Continuous live video feed.** Snapshot-on-command only — privacy and latency.

How Claude Code should use this file

1. **Match `CONTRACT.md` exactly** — never invent endpoint signatures; mark anything unverified `[CONFIRM]`.
2. **Build the feature, not just the endpoint** — each item above has a *behavior* and a *novel mechanic*. The acceptance ("Done") is the spec; the ★ items must be unmistakable in the demo.
3. **Map:** every feature here → an endpoint in `CONTRACT.md` → files/phases in `CLAUDE.md` → a slot in `PLAN.md`.
4. **Living docs:** as you build, keep your `packages/<track>/CLAUDE.md` and `STATUS.md` current; log anything faked under "Faked / TODO real." A change isn't done until the docs reflect it.
5. **Order of effort:** ★ differentiators + ★ core first → supporting → exactly one optional. Do not start a feature on the ✗ list.